

Multi-Agent AI Day Planner

Technical Specification & Project Definition

June 2026

1. Overview

1.1 Project type

Dimension	Value
Project category	Multi-agent AI system + RAG knowledge assistant
Primary interface	Streamlit web UI + REST API (FastAPI)
Deployment target	Azure Container Apps (Bicep IaC + GitHub Actions CI/CD)
Cost model	Cost-aware agent routing — sub-\$0.01 per user per day
Agents	OpenAI gpt-4o (paid) · Groq llama-3.1-8b-instant (free) · Ollama mistral (local, free)

1.2 Problem statement

Professionals lack a unified AI system that intelligently plans their entire day — from wake-up to sleep — incorporating personalised diet plans, structured workout and yoga sessions, BMI-based weight management, and proactive reminders via SMS and app notifications.

Existing wellness tools are either too generic to be actionable, prohibitively expensive, or require stitching together multiple disconnected applications with no unified intelligence layer. There is no single system that orchestrates planning, nutrition, fitness, and health analytics through a cost-aware multi-agent architecture.

1.3 Target users

- Working professionals who need structured daily planning across health, nutrition, and fitness
- Fitness-conscious individuals seeking weight management, macro tracking, and consistent workout scheduling

1.4 Tech stack

Layer	Technologies
Backend & AI	FastAPI · LangGraph · OpenAI gpt-4o · Groq llama-3.1-8b-instant · Ollama mistral
Frontend	Streamlit 1.41 — 5-page web UI with sidebar navigation
Vector store	Pinecone (Azure eastus, 1536 dims, cosine) · FAISS local fallback
State & scheduling	Redis 7 · APScheduler (AsyncIOScheduler) · RedisJobStore
Observability	Prometheus · Grafana 10.2 · Loki 2.9 · Promtail
Infrastructure	Docker Compose · Azure Container Apps · ACR · Bicep IaC

Layer	Technologies
CI/CD	GitHub Actions — test → build → push ACR → deploy Bicep on every push to main
Notifications	Twilio (primary) · Fast2SMS (fallback)

2. Goals

G1 — Cost-aware multi-agent orchestration via LangGraph

Builds a state machine with five nodes routing each task to the cheapest capable agent. A static routing table maps 10 task types to OpenAI, Groq, or Ollama. Any unknown task type defaults to the free local Ollama agent.

G2 — Personalised daily schedule generation

Generates a full 24-hour timeline from configurable wake and sleep times. Each scheduled event carries a time, category, duration, and a ready-to-send SMS reminder message within 160 characters.

G3 — Meal planning with automatic cheat meal detection

Generates breakfast, lunch, and dinner for any given date. Automatically switches to cheat-meal mode on Saturdays and Sundays. Each meal includes name, calories, prep time, and ingredients.

G4 — Proactive SMS reminders via APScheduler and Redis

Registers one reminder job per plan event in a Redis-backed job store. Each job fires 10 minutes before the event. Jobs survive API restarts. Past events are silently skipped.

G5 — Dual SMS provider with automatic failover

Checks for a Twilio key first; falls back to Fast2SMS if not configured. If neither is set, logs a warning and continues without crashing. Provider selection is fully automatic from environment variables.

G6 — BMI calculation and AI 4-week weight plan

Instant BMI calculation using the WHO formula with no AI call. Based on the result, categorises the user as Underweight, Normal, Overweight, or Obese and sets the appropriate goal. An AI-generated 4-week plan then provides calorie targets, macros, a weekly workout schedule, food guidance, and week-by-week milestones.

G7 — RAG knowledge assistant: 4-stage pipeline

A complete retrieve-rerank-generate-verify pipeline. The query is embedded and matched against a Pinecone vector store, top results are re-ranked by a fast Groq model, OpenAI generates the final answer, and a second Groq call checks every claim against the source chunks for hallucinations.

G8 — Plan history with 90-day rolling retention

Every generated plan is automatically saved to Redis per user. Retention is capped at 90 entries per plan type with a 90-day TTL. No client action is required — saving happens inside every generate endpoint before returning the response.

3. Non-Goals

NG1 — No embedded workout or yoga video clips

The current planner generates text-based workout and yoga schedules only. Embedding short video clips or animated exercise demonstrations directly inside the Streamlit UI would require a media hosting service, a content delivery network, licensing agreements for exercise footage, and a video rendering layer that Streamlit does not natively support. Serving proprietary or third-party fitness clips also introduces copyright and storage cost concerns. This is deferred to v2, where a dedicated media microservice could serve pre-approved clips linked to each scheduled activity.

NG2 — No inline calorie calculator for recommended foods

The meal planner currently provides estimated calorie counts per meal generated by the AI model. A proper calorie calculator — one that lets users adjust portion sizes, swap ingredients, and get recalculated totals in real time — requires a structured nutritional database (such as USDA FoodData Central or Nutritionix), a food-matching API, and a stateful UI component to track cumulative intake across meals. Building this accurately goes significantly beyond prompt engineering into data integration and UI state management, and is out of scope for v1.

NG3 — No YouTube video recommendations for meals and workouts

The agent does not currently call the YouTube Data API to surface relevant cooking tutorials or workout videos. Integrating YouTube recommendations requires a separate API key with quota management, a search query construction layer per meal and exercise, and result ranking logic to surface high-quality content. There are also content-moderation considerations around surfacing user-generated videos to end users. This is a well-defined v2 feature — the agent already generates meal names and workout types that could be used directly as YouTube search queries, making the integration straightforward once the API access is provisioned.

4. Success Metrics

Metric	Target
Plan generation latency p95	< 10 seconds
Cost per user per day	< \$0.01
SMS delivery rate	> 95 %
RAG hallucination flag rate	< 5 %
API uptime (Azure)	> 99.5 %
Scheduler reliability (jobs fired on time)	> 99 %
API request error rate	< 1 %
OpenAI share of total agent calls	≤ 30 %

5. Feature Definitions

5.1 — Orchestrator

Agent / Component: LangGraph — routes to OpenAI · Groq · Ollama

How it works

A LangGraph state machine receives every API request and routes it to the cheapest capable agent based on a static routing table. The state object carries the task type, user input, context, and the result from whichever agent responds. All three agents — OpenAI, Groq, and Ollama — are wired as separate nodes. The graph aggregates whichever node produces a result and returns it.

Agent capabilities

OpenAI handles all high-quality generation tasks: schedules, meals, and weight plans. Groq handles speed-critical tasks: RAG re-ranking, hallucination verification, and book recommendations at no cost. Ollama handles summarisation, evaluation, and acts as the zero-cost fallback for any unrecognised task type.

Observability

Every agent call increments a Prometheus counter and records latency in a histogram. The middleware layer records every API request with method, path, status code, and elapsed time — all shipped to Loki for log querying in Grafana.

5.2 — Day Planner

Agent / Component: OpenAI gpt-4o

Schedule generation

Accepts configurable wake and sleep times plus optional freeform preferences. Calls OpenAI once per day to produce a structured timeline of events. Fixed rules are enforced in the prompt: 45-minute workout on Monday, Wednesday, and Friday; yoga or mobility on other days; 10-minute evening meditation daily; 30-minute book reading before bed.

Event structure

Each event in the timeline contains a 24-hour time, activity description, duration in minutes, a category tag (sleep, wake, workout, meditation, meal, reading, work, or break), and a pre-composed SMS reminder message within 160 characters.

Reminder registration

If a phone number is provided, the system automatically registers one APScheduler job per event at plan generation time. Each job fires 10 minutes before the event time. Jobs are stored

in Redis so they survive server restarts. Past-time events are silently skipped. The response includes a count of reminders successfully scheduled.

5.3 — Meal Planner

Agent / Component: OpenAI gpt-4o

Daily meal plan

Generates breakfast, lunch, and dinner for a given date. Each meal includes a name, estimated calorie count, preparation time in minutes, and a full ingredient list. The prompt instructs the model to balance macros, minimise processed ingredients, and vary cuisine style across the week.

Cheat meal detection

Automatically detects Saturdays and Sundays and switches the prompt to cheat-meal mode, suggesting a popular street food or restaurant-style dish instead of a balanced meal. The response includes an `is_cheat_day` flag so the UI can display it distinctly.

History

Every meal plan is saved to the Redis history store immediately after generation under the meals plan type, scoped to the requesting user ID.

5.4 — BMI & Weight Management

Agent / Component: Pure math (BMI) · OpenAI gpt-4o (plan)

BMI calculation

An instant calculation endpoint that requires no AI call. Accepts weight in kilograms and height in centimetres, computes BMI using the standard WHO formula, and returns the value, category (Underweight / Normal weight / Overweight / Obese), recommended goal (lose, gain, or maintain), ideal weight, and kilograms to change.

4-week weight plan

A full user profile — name, age, gender, weight, height, and activity level — is sent to OpenAI. The model returns a daily calorie target, protein, carbohydrate, and fat targets in grams, a 7-day workout schedule with intensity levels, food guidance (what to favour and what to avoid), week-by-week milestone expectations, and a personalised motivational note.

Goal routing

BMI below 18.5 triggers a caloric-surplus gain plan focused on strength. BMI in the normal range triggers a body-recomposition maintenance plan. BMI of 25 or above triggers a caloric-deficit loss plan focused on cardio. The goal is set automatically based on the calculated BMI.

5.5 — RAG Knowledge Assistant

Agent / Component: Groq (query/rerank/verify) · OpenAI (answer)

Stage 1 — Retrieval

The user question is converted to a 1536-dimensional vector using the OpenAI text-embedding-3-small model. The vector is queried against a Pinecone index hosted on Azure (easus, cosine similarity) or a local FAISS index as a fallback, returning the top 10 matching chunks.

Stage 2 — Re-ranking

Groq scores each of the 10 retrieved chunks for relevance to the original question. The top 5 highest-scoring chunks are selected and passed to the answer generator. If the re-ranking response cannot be parsed, the original retrieval order is preserved.

Stage 3 — Answer generation

OpenAI synthesises a final answer from the top-5 ranked chunks with a grounded prompt that prohibits going beyond the provided context. The response includes the answer text and a list of cited sources.

Stage 4 — Hallucination verification

A second Groq call independently checks every claim in the generated answer against the source chunks. It returns a boolean flag, a confidence score between 0 and 1, and a list of any unsupported claims. This result is always included in the API response so callers can decide how to handle flagged answers.

5.6 — SMS Reminder System

Agent / Component: APScheduler · Redis · Twilio / Fast2SMS

Scheduling

When a day plan is generated with a phone number, the system iterates every event in the plan, computes a reminder time 10 minutes before the event, and registers a date-triggered job in APScheduler backed by a Redis job store. Jobs are idempotent — re-generating a plan for the same date and phone replaces existing jobs rather than duplicating them.

Delivery

At firing time, the job calls the SMS delivery function which checks for a Twilio configuration first, then falls back to Fast2SMS. If neither provider is configured, the failure is logged without crashing the scheduler. Both providers return a boolean success flag; errors are caught and logged rather than raised.

5.7 — Plan History

Agent / Component: Redis — no agent

Automatic saving

Every successful plan generation endpoint — day plan, meals, and weight plan — saves the full response payload to Redis before returning it to the client. No explicit save action is needed from the UI or API caller.

Retention and retrieval

History is stored per user per plan type. A rolling window keeps the newest 90 entries per combination, and a 90-day TTL ensures automatic expiry. The history endpoint accepts an optional plan type filter and a configurable result limit between 1 and 50.

Deletion

A delete endpoint accepts an optional plan type. Without a plan type, it clears all history for that user. With a plan type, it clears only that category. The response returns the count of Redis keys deleted.

5.8 — Observability

Agent / Component: Prometheus · Grafana · Loki · Promtail

Prometheus metrics

Five metrics are collected: a counter for total agent calls labelled by agent name, a latency histogram per agent with seven buckets from 100ms to 30 seconds, a counter for API requests labelled by method, endpoint, and status code, a counter for SMS reminders sent labelled by provider, and a gauge for active users.

Structured log shipping

A custom Python logging handler batches log entries every 2 seconds and ships them as structured JSON to Grafana Loki via HTTP. The handler uses a bounded in-memory queue so log emit calls never block the async event loop. Network failures in the shipping thread are silently swallowed — logging never crashes the application.

Grafana dashboard

A pre-built dashboard with five panels is provisioned automatically at container startup: agent calls per minute by agent, agent latency p95 by agent, total SMS reminders sent, API requests per minute by endpoint, and a live Loki log stream filtered to the application label. Both Prometheus and Loki are registered as datasources via provisioning config mounted into the Grafana container.

Log collection

Promtail scrapes stdout from all Docker containers using the Docker socket, applies container name and Compose service labels, and pushes entries to Loki every 5 seconds. No application-level log files are written to disk.

6. UI Mock

The following screenshot shows the live Streamlit application running at localhost:8501. The Day Planner page is active, showing the sidebar navigation (Day Planner, Meal Plan, BMI & Weight, History, Analytics), the wake-up and sleep time pickers, a freeform preferences field, and the Generate My Day Plan action button. The User ID field is populated with user_001 and the Phone (SMS) field is visible below the sidebar.

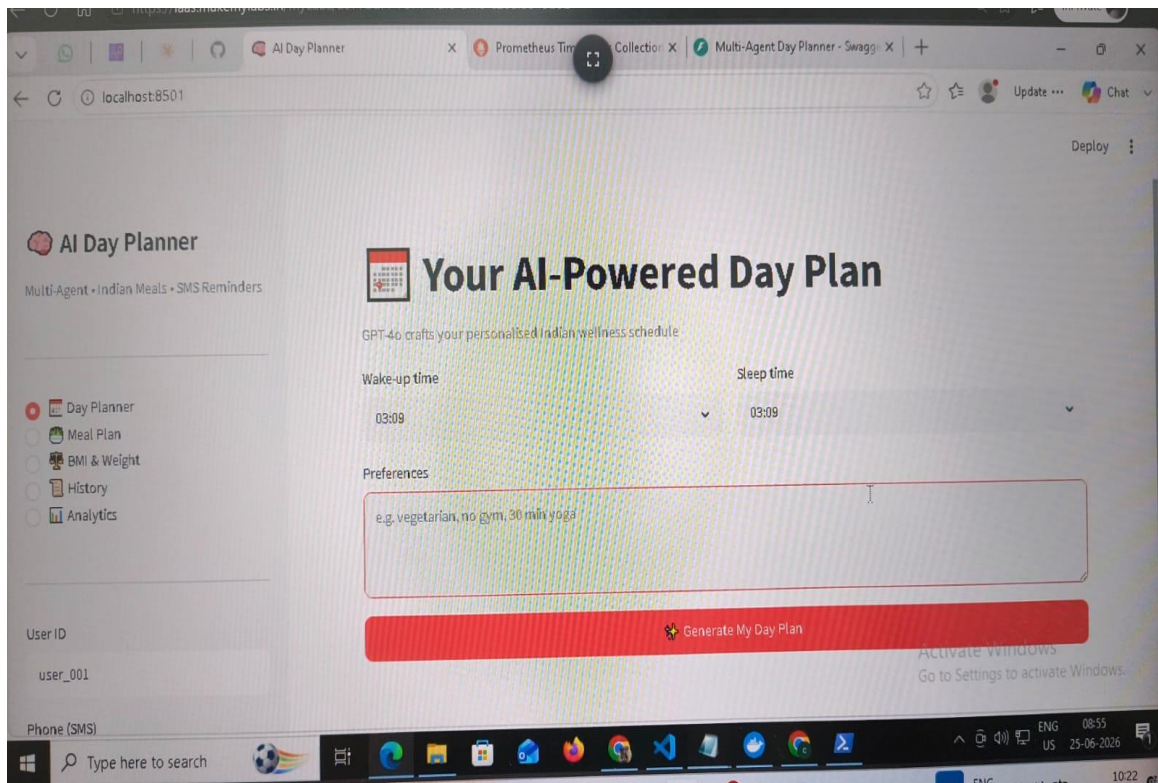


Figure 1 — Streamlit Day Planner UI running at localhost:8501 (25 June 2026)

7. Azure Deployment

The entire infrastructure is defined as code using Azure Bicep (`azure/main.bicep`) and deployed via a single GitHub Actions workflow on every push to the main branch. The workflow runs unit tests first, then provisions infrastructure, builds both Docker images in Azure Container Registry, and deploys them to Container Apps.

7.1 Infrastructure summary

Resource	Purpose
Azure Container Registry	Stores the FastAPI and Streamlit Docker images built by the CI pipeline
Azure Cache for Redis	Hosts the APScheduler job store and 90-day plan history; TLS-only, Basic C0 tier
Log Analytics Workspace	Receives container logs from the Container Apps environment; 30-day retention
Container Apps Environment	Shared managed environment for both application containers
FastAPI Container App	Runs the API — 0.5 vCPU, 1 GiB RAM, external HTTPS ingress, scales 1 to 5 replicas
Streamlit Container App	Runs the UI — 0.25 vCPU, 0.5 GiB RAM, external HTTPS ingress, scales 1 to 3 replicas

7.2 Secrets and configuration

API keys (OpenAI, Groq) are passed as secure Bicep parameters and stored as Container App secrets. They are never written to ARM output or environment files. The Redis connection string is constructed dynamically at deploy time from the Redis primary key and stored the same way. Each container references secrets by name rather than injecting values directly into environment variables.

7.3 CI/CD pipeline

- Trigger: push to the main branch
- Step 1: Run unit tests — pipeline stops on failure
- Step 2: Azure login and Bicep deployment — infrastructure is idempotent
- Step 3: Build and push FastAPI image to ACR tagged with the Git commit SHA
- Step 4: Build and push Streamlit image to ACR tagged with the Git commit SHA